

CPF | Core Platform Framework

Business Platform Documentation

CPF 배치 개발자 가이드

Batch Job 설계부터 분산 실행·복구·운영 인계까지 필요한 계약을 선택합니다.

문서 기준

Documentation Harness v2.1.0

기준 **Source**

master 054d894b47f4be8323439dc6f9e58b7d8b60fe54

기준일

2026-08-26

이 문서는 최신 Source 의 공개 계약과 운영 경계를 기준으로 작성하며, 실행하지 않은 검증을 성공으로 기록하지 않습니다.

목차

역할과 질문을 기준으로 필요한 절을 빠르게 찾습니다.

1. Batch 구조와 기능 선택

- 1.1 Job/Step/Chunk/Partition 선택
- 1.2 Control Plane · Scheduler · Worker · Center-Cut · Agent 역할
- 1.3 개발용 Local Runtime 과 운영 Process 구분

2. Job · Step · Chunk · Partition 개발

- 2.1 Job/Step 작성
- 2.2 Chunk 와 Transaction
- 2.3 Partition 과 Execution Context
- 2.4 Parameter · Restartability

3. 상태 · Idempotency · 동시성

- 3.1 Execution/Checkpoint 상태
- 3.2 Idempotency
- 3.3 Lease · Fencing · Heartbeat
- 3.4 중복 실행 방지

4. Control Plane 과 실행 Runtime

- 4.1 Control Plane · Scheduler
- 4.2 Worker
- 4.3 Center-Cut
- 4.4 Agent · Job Pack · 배포

5. 복구 방법 선택

5.1 Restart · Rerun · Reprocess · Reconcile

5.2 Process Kill 후 복구

5.3 부분 실패

5.4 Drain · Resume · Rollback

6. 외부 Side Effect 와 UNKNOWN

6.1 외부 호출/메시지 Side Effect

6.2 UNKNOWN

6.3 Reconcile · Compensation

6.4 재처리 안전성

7. 대량 처리와 성능

7.1 Partition/Chunk 크기

7.2 Center-Cut 대상 분할

7.3 처리량 · Lock · DB 영향

7.4 다중 Worker 용량

8. Test · 운영 인계 · Source

8.1 Lifecycle Test

8.2 Process Kill/Recovery Test

8.3 운영 인계

8.4 금지 패턴 · Source 길찾기

1. Batch 구조와 기능 선택

그림 1-1. Batch Control & Execution



Control Plane 과 실행 Lane 을 분리하고 Lease·Fencing·Checkpoint 로 실행 소유권을 보호합니다.

1.1 Job/Step/Chunk/Partition 선택

Job 은 업무 실행 단위, Step 은 처리 단계입니다. 대량 레코드는 Chunk, 독립 분할이 가능한 대상은 Partition 을 선택하며 checkpoint 와 transaction 경계를 같은 처리 의미로 맞춥니다.

- Partition key 는 재시작 시 동일 대상을 재구성할 수 있어야 합니다.
- 대량 처리에서 chunk 크기와 DB lock/commit 비용을 함께 측정합니다.

1.2 Control Plane·Scheduler·Worker·Center-Cut·Agent 역할

Control Plane 은 제어 정책과 상태를, Scheduler 는 실행 계획을, Worker 는 실제 Job 실행을 담당합니다. Center-Cut 은 대상 분할·집계를, Agent 는 Job Pack/Process 실행 경계를 담당합니다. Batch 내부 Kafka/Broker Remote Execution 은 사용하지 않습니다.

- Scheduler 는 실행 계획/trigger 를, Worker 는 실제 Job 실행을 소유합니다.
- Center-Cut/Agent 는 대상 분할·Process 실행과 lease/fencing 상태를 추적합니다.

1.3 개발용 Local Runtime 과 운영 Process 구분

Local Runtime 은 개발·테스트 편의를 위한 실행 형태이고 운영에서는 Scheduler·Worker·Center-Cut·Agent 를 실제 Process 와 instance identity 로 구분합니다. Local 성공만으로 운영 topology 를 검증한 것으로 보지 않습니다.

하려는 일에 맞는 CPF 기능과 선택 조건을 고릅니다.

각 역할의 책임과 장애 시 영향을 구분합니다.

표 1-1. 역할 / 책임 / 소유/경계 / 장애 영향

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

역할	책임	소유/경계	장애 영향
Control Plane	정의·제어·상태	제어 Owner	명령/상태 불일치
Scheduler	Trigger·할당	실행 계획	중복 scheduling
Worker	Job 실행	execution	Process kill
Center-Cut	대상 분할·조정	대량 실행	부분 실패
Agent	Job Pack/Process 실행 · heartbeat	instance/lease	stale agent

Source [길찾기 cpf-batch/runtime/src/main/java/com/cpf/batch/execution/CpfBatchExecutionAutoConfiguration.java](#)

2. Job·Step·Chunk·Partition 개발

2.1 Job/Step 작성

Job 은 입력 Parameter 와 restartability 를 정의하고 Step 은 처리 순서와 실패 경계를 명확히 합니다. Step 간 데이터는 명시적 Execution Context 나 영속 상태로 전달합니다.

2.2 Chunk 와 Transaction

Chunk 는 읽기·처리·쓰기 묶음과 commit/checkpoint 경계를 결정합니다. Chunk 크기는 처리량뿐 아니라 DB lock, rollback 범위와 메모리 사용량을 함께 고려합니다.

- 대량 처리에서 chunk 크기와 DB lock/commit 비용을 함께 측정합니다.

2.3 Partition 과 Execution Context

Partition 은 동일 Job 대상을 안정적인 key 로 분할하고 각 실행의 Execution Context 를 독립적으로 추적합니다. 재시작 시 동일 partition 을 재구성할 수 있어야 합니다.

- Partition key 는 재시작 시 동일 대상을 재구성할 수 있어야 합니다.

2.4 Parameter·Restartability

Job Parameter 는 재실행 시 같은 업무 대상을 식별할 수 있도록 안정적으로 정의하고 restartable 여부를 Job 의미에 맞게 고정합니다. 변경 가능한 운영 값과 실행 identity 값을 구분합니다.

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기본값이 아닌 실제 선택이 필요한 옵션을 확인합니다.

Source [길찾기 cpf-batch/api/src/main/java/com/cpf/batch/api/BatchParameterDefinition.java](#)

3. 상태·Idempotency·동시성

3.1 Execution/Checkpoint 상태

Execution 상태와 Checkpoint 는 재시작 가능 지점을 결정하는 영속 계약입니다. Process 메모리만으로 완료 여부를 판단하지 않고 실행 ID 와 상태 전이를 기준으로 복구합니다.

3.2 Idempotency

Idempotency 는 동일 의미의 요청이 재전송돼도 Side Effect 가 한 번만 확정되도록 durable 상태로 관리합니다. 메모리 flag 만으로 다중 인스턴스 중복을 막지 않습니다.

- key scope 와 결과 보존 기간을 업무 Operation 과 맞춥니다.
- 처리 중 Process Kill 후에도 재시도가 같은 결과를 재사용하거나 안전하게 복구해야 합니다.

3.3 Lease·Fencing·Heartbeat

Lease 는 작업 소유권의 유효기간이고 Fencing 은 stale Worker 의 늦은 쓰기를 차단하는 토큰입니다. Heartbeat 만 살아 있다고 소유권을 무기한 인정하지 않습니다.

- lease 만료 후 재할당 시 fencing token 이 증가해야 합니다.
- clock/timeout 경계와 DB update 조건을 동시성 Test 로 검증합니다.

3.4 중복 실행 방지

중복 실행 방지는 Job/업무 key 와 실행 상태를 durable 하게 비교해 같은 대상의 동시 claim 을 차단합니다. 단일 Worker 의 메모리 flag 만으로는 다중 인스턴스 중복을 막을 수 없습니다.

- 중복 실행 key 는 Job·업무 대상·실행 정책 범위를 구분해 동일 대상의 동시 claim 을 막습니다.
- Process Kill 뒤 stale lease/fencing token 으로 재진입한 Worker 의 쓰기를 거절해야 합니다.

현재 상태에서 허용되는 다음 처리와 금지를 확인합니다.

상황에 맞는 선택과 피해야 할 경로를 판단합니다.

[Source 길찾기](#) `cpf-batch/runtime/src/main/java/com/cpf/batch/execution/CpfBatchIdempotencyConflictException.java`

4. Control Plane 과 실행 Runtime

4.1 Control Plane·Scheduler

Control Plane 은 Job 실행 정책·제어 명령을 소유하고 Scheduler 는 실행 시점과 trigger 를 계산합니다. 계획 생성과 실제 Worker 실행 결과를 같은 상태로 취급하지 않습니다.

4.2 Worker

Worker 는 할당된 Job/Step 을 실행하고 heartbeat·lease·checkpoint 를 갱신합니다. Worker 장애 후 다른 Worker 가 이어받을 수 있도록 Process-local 상태에만 의존하지 않습니다.

4.3 Center-Cut

Center-Cut 은 대량 대상의 분할·진행·집계를 관리합니다. 일부 대상 실패를 전체 성공으로 덮지 않고 실패 범위와 재처리 대상이 식별되게 합니다.

4.4 Agent·Job Pack·배포

Agent 는 Job Pack/Process 실행 경계를 담당하고 manifest/version 과 실행 결과를 연결합니다. Batch 작업 분배는 DB claim·lease·fencing 을 사용합니다. 잘못된 Job Pack 이나 stale version 을 조용히 실행하지 않습니다.

각 역할의 책임과 장애 시 영향을 구분합니다.

표 4-1. 역할 / 책임 / 소유/경계 / 장애 영향

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

역할	책임	소유/경계	장애 영향
Control Plane	정의·제어·상태	제어 Owner	명령/상태 불일치
Scheduler	Trigger·할당	실행 계획	중복 scheduling
Worker	Job 실행	execution	Process kill
Center-Cut	대상 분할·조정	대량 실행	부분 실패
Agent	Job Pack/Process 실행 ·heartbeat	instance/lease	stale agent

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

Source [깃찾기](#) [cpf-batch/control-plane/src/main/java/com/cpf/batch/control/deploy/DeploymentEngine.java](#)

5. 복구 방법 선택

그림 5-1. Recovery State



결과 불명확 상태를 UNKNOWN 으로 보존하고 Reconcile 후 안전한 재처리를 선택합니다.

5.1 Restart·Rerun·Reprocess·Reconcile

Restart 는 중단된 실행을 checkpoint 에서 이어가고, Rerun 은 실행 정의를 새로 수행하며, Reprocess 는 업무 대상을 다시 처리합니다. 결과 불명확 상태는 Reconcile 로 먼저 확정합니다.

- UNKNOWN 에서 무조건 재호출하면 중복 Side Effect 가 생길 수 있습니다.
- Reconcile 결과·actor·reason·시각을 추적 가능하게 남깁니다.

5.2 Process Kill 후 복구

Process Kill 후에는 마지막 durable checkpoint, lease/fencing, 외부 Side Effect 를 확인한 뒤 Restart 또는 Reconcile 경로를 선택합니다.

5.3 부분 실패

부분 실패는 성공한 partition/chunk 와 실패한 범위를 분리해 기록하고, 재처리 시 이미 확정된 Side Effect 를 다시 만들지 않도록 대상과 idempotency 를 제한합니다.

5.4 Drain·Resume·Rollback

Drain 은 신규 할당을 멈추고 in-flight 실행을 안전 상태로 이동시킵니다. Resume 은 lease/ownership 을 확인한 뒤 재개하고, 배포 Rollback 은 Job Pack·schema·상태 compatibility 를 확인합니다.

- Drain 완료 전 신규 claim 이 중단되고 in-flight execution 이 안전 지점에 도달했는지 확인합니다.
- Rollback 은 Job Pack·설정·schema compatibility 와 이미 발생한 업무 Side Effect 를 분리해 판단합니다.

Restart·Rerun·Reprocess·Reconcile 의 목적과 위험을 구분합니다.

표 5-1. 판단 기준 / Restart / Rerun / Reprocess / Reconcile

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

판단 기준	Restart	Rerun	Reprocess	Reconcile
핵심 목적	중단 지점 재개	실행 정의 다시 수행	업무 대상 다시 처리	불명확 결과 확정
선행 확인	checkpoint	중복 Side Effect	대상 범위/idempotency	외부 실제 상태
주요 위험	stale lease	중복 실행	중복 업무 반영	근거 없는 상태 확정

Drain·Resume·Rollback 각각의 실행 조건과 완료 확인 기준을 비교합니다.

Source [길찾기](#) `cpf-batch/control-plane/src/main/java/com/cpf/batch/control/deploy/DeploymentRecoveryController.java`

6. 외부 Side Effect 와 UNKNOWN

6.1 외부 호출/메시지 Side Effect

Batch 외부 호출이나 메시지 발행은 DB commit 과 별도 Side Effect 로 추적합니다. 응답 유실이나 broker 상태 불명확 시 성공/실패를 추정하지 않고 UNKNOWN 과 Reconcile 을 사용합니다.

6.2 UNKNOWN

UNKNOWN 은 결과를 확정할 Evidence 가 부족한 정상적인 운영 상태입니다. Reconcile 은 외부기관·DB·Broker 의 실제 결과를 조회해 상태를 확정하거나 manual review 로 넘깁니다.

- UNKNOWN 에서 무조건 재호출하면 중복 Side Effect 가 생길 수 있습니다.
- Reconcile 결과·actor·reason·시각을 추적 가능하게 남깁니다.

6.3 Reconcile·Compensation

분산 Side Effect 는 Local Transaction 으로 감출 수 없습니다. Saga/TCC/XA 는 참여 Resource 와 복구 모델에 따라 선택하고, 결과 불명확 시 보상보다 먼저 Reconcile 가능성을 판단합니다.

- 동일 요청의 재시도는 idempotency key 와 상태 저장으로 안전하게 만듭니다.
- Compensation 은 이미 확정된 Side Effect 를 되돌리는 별도 업무 동작으로 추적합니다.

6.4 재처리 안전성

재처리는 이미 처리된 업무 대상의 Side Effect 를 다시 만들 수 있으므로 대상 key, 기존 결과, idempotency 와 compensation 가능성을 확인한 뒤 제한된 범위로 수행합니다.

오류 상태에서 Retry·복구·확인 순서를 판단합니다.

현재 상태에서 허용되는 다음 처리와 금지를 확인합니다.

Source [길찾기](#) `cpf-batch/api/src/main/java/com/cpf/batch/api/BatchExecutionControlPort.java`

7. 대량 처리와 성능

7.1 Partition/Chunk 크기

Partition/Chunk 크기는 처리량뿐 아니라 DB lock·commit 비용, 메모리, retry 범위와 Worker 수를 함께 측정해 정합니다.

- partition key 는 재시작 시 동일 대상을 재구성할 수 있어야 하고 skew 가 심하면 분할 전략을 조정합니다.
- chunk 크기는 rollback 범위와 commit/lock 비용을 실제 데이터량으로 측정해 결정합니다.

7.2 Center-Cut 대상 분할

Center-Cut 분할은 대상 수, key 분포, 실패 격리와 재처리 단위를 기준으로 설계합니다. 분할 수를 늘리는 것만으로 처리량이 선형 증가한다고 가정하지 않습니다.

7.3 처리량·Lock·DB 영향

Lock 은 동일 Resource 에 대한 동시 Side Effect 를 제어하는 수단입니다. DB lock, optimistic concurrency, distributed lock 중 소유 범위와 장애 모델에 맞는 방식을 선택합니다.

- lease 기반 lock 은 만료와 fencing token 을 함께 검토합니다.
- lock 대기시간을 transaction timeout 보다 길게 두지 않습니다.

7.4 다중 Worker 용량

다중 Worker 용량은 CPU 보다 DB connection·lock, partition backlog, lease/heartbeat 비용, lease/heartbeat 비용까지 함께 측정합니다. scale-out 시 duplicate claim 이 없는지 검증합니다.

상황에 맞는 선택과 피해야 할 경로를 판단합니다.

Oracle·PostgreSQL·MariaDB 의 공통 의미와 검증 지점을 비교합니다.

표 7-1. 항목 / Oracle / PostgreSQL / MariaDB

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	Oracle	PostgreSQL	MariaDB
공식 지원	지원	지원	지원
Canonical 의미	공통	공통	공통
Vendor Render	oracle	postgresql	mariadb
검증	DDL·Runtime	DDL·Runtime	DDL·Runtime

Source [길찾기 cpf-batch/worker/src/main/java/com/cpf/batch/worker/CpfBatchWorkerIdentity.java](https://github.com/cpf-batch/worker/src/main/java/com/cpf/batch/worker/CpfBatchWorkerIdentity.java)

8. Test·운영 인계·Source

8.1 Lifecycle Test

Batch Lifecycle Test 는 Fresh 상태→실행→checkpoint→Process

Kill→Restart/Rerun/Reprocess/Reconcile→Cleanup→재실행까지 전체 흐름을 검증합니다.

- 실행하지 않은 Test 는 PASS 로 기록하지 않습니다.
- Source identity, 명령, ExitCode, Evidence 를 같은 실행 결과에 연결합니다.

8.2 Process Kill/Recovery Test

Recovery Test 는 실행 중 Process 를 실제로 종료한 뒤 lease 만료·재할당·checkpoint 복구·중복 Side Effect 부재까지 확인합니다.

8.3 운영 인계

운영 인계에는 Job/Execution ID, Scheduler/Worker instance, checkpoint, lease/fencing, 외부 Side Effect 와 재처리 기준을 Runbook 과 함께 전달합니다.

8.4 금지 패턴·Source 길찾기

금지 패턴은 Owner·Topology·신뢰 경계를 우회하거나 운영 Evidence 를 잃게 만드는 구현을 빠르게 차단하기 위한 Review Gate 입니다.

- self-HTTP, 타 Owner DB/Internal 직접 참조, secret 원문 로그를 금지합니다.
- 미실행 검증·stale Evidence·Optional 기능의 숨은 Side Effect 를 완료 근거로 사용하지 않습니다.

정상·실패·복구까지 통과해야 할 시나리오를 확인합니다.

계약과 실제 Consumer 를 확인할 Source 위치를 찾습니다.

Source 길찾기 [cpf-tools/verification/tools/run-cpf-development-final-gate.py](https://github.com/harness/cpf-tools/verification/tools/run-cpf-development-final-gate.py)